

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 102. Indexing and Slicing - Read

Section 1: Lecture Summary

This lecture introduces **indexing and slicing** operations on Python lists, focusing initially on reading (retrieving) elements. Indexing allows access to a single element by its position, using positive or negative indices. Slicing extracts a sub-list using a range defined by start, stop, and optional step values. The lecture explains the syntax and behavior of slicing, including handling defaults (omitted start or stop), and the step parameter for skipping elements or reversing the list. Examples demonstrate forward and backward traversal with positive and negative steps. The lecture emphasizes that these operations return new list data (copies) and can be assigned to variables. The mutable nature of lists is mentioned, with the write (modification) aspect of indexing and slicing reserved for the next lecture.

Section 2: Key Concepts and Explanations

- List Mutability

- Lists are mutable: elements can be read and modified.
- Indexing and slicing support both read and write operations.
- This lecture covers reading only; writing is in the next lecture.

- Indexing

- Access a single element by position using square brackets `list[index]`.
- Positive indices: start at 0 for the first element.
- Negative indices: start at -1 for the last element, counting backward.
- Example: `list[4]` or `list[-3]` both return the element at that position.
- Result of indexing is a single value, which can be assigned to a variable.

- Slicing

- Syntax: `list[start:stop:step]` - square brackets with colons separate up to three parts.
- All three parts are optional:

- start: index to begin slice (default start of list if omitted).
- stop: index to end slice (exclusive; defaults to end of list if omitted).
- step: increment between indices (default is 1).
- Slice returns a new list (not modifying original).
- The stop index is exclusive, meaning elements are included only up to one before the stop index.
- Negative indices can be used for start, stop, and step.
- Step controls direction:
 - Positive step: slice goes forward.
 - Negative step: slice goes backward, requiring start > stop.
- **Examples of Slicing Behavior**
 - `list[:]` or `list[::]` returns a full copy of the list.
 - `list[2:]` returns elements from index 2 to end.
 - `list[2:5]` returns elements index 2 to 4.
 - `list[-5:-2]` returns elements with negative indices equivalent to 2:5.
 - Steps control skipping and order:
 - `list[::2]` returns every second element (step 2).
 - `list[::-1]` returns the whole list reversed.
 - When using a negative step, start should be greater than stop, slicing backwards.

- **Assignment of Results**

- Both indexing and slicing results can be assigned to variables for further use.

Section 3: Example Code and Use Cases

Creating a list

Indexing examples (reading)

Assigning indexed value to variable

Slicing examples (reading)

Negative step with specified start and stop

```
l1 = [3, 6, 9, 12, 15, 18, 21]

print(l1[4])      # Output: 15 (element at index 4)
print(l1[-3])     # Output: 15 (element 3rd from the end)

x = l1[-3]
print(x)          # Output: 15

print(l1[:])      # Output: [3, 6, 9, 12, 15, 18, 21] (whole list)
print(l1[2:])     # Output: [9, 12, 15, 18, 21] (from index 2 to end)
print(l1[2:5])    # Output: [9, 12, 15] (from index 2 to 4)
print(l1[-5:-2])  # Output: [9, 12, 15] (same as above using negative
indices)

print(l1[::1])    # Output: [3, 6, 9, 12, 15, 18, 21] (step 1, full list)
print(l1[::2])    # Output: [3, 9, 15, 21] (every second element)
print(l1[::3])    # Output: [3, 12, 21] (every third element)

print(l1[::-1])   # Output: [21, 18, 15, 12, 9, 6, 3] (list reversed)

print(l1[4:0:-1]) # Output: [15, 12, 9, 6] (from index 4 down to 1 in
reverse)
print(l1[-3:-7:-1]) # Output: [15, 12, 9, 6] (same as above using negative
indices)
```

Explanation: These code snippets show accessing and extracting elements by index and slice, including examples with positive and negative indices and steps. They also illustrate how results can be assigned to variables or printed directly.

Section 4: Key Takeaways

- Python lists support **indexing** for single element access using positive or negative indices.
- Python lists support **slicing** to extract sub-lists, using `[start:stop:step]` syntax.

- All parts of slicing—start, stop, and step—are optional; defaults apply when omitted.
- The `stop` index in slicing is exclusive; elements are included up to `stop - 1`.
- The `step` parameter controls skipping and direction; negative step reverses the traversal.
- Indexing and slicing operations return new data and do not modify the original list (for reading).
- Results of indexing or slicing can be stored in variables.
- Lists are mutable, so indexing/slicing can be used to modify data as well, but writing is covered separately.
- Understanding indexing and slicing on lists extends naturally to other data structures with similar operations (like strings).